

Lab 3 — Trees, Random Forests and Gradient Boosting

Machine Learning I · Tutorial

Rodrigo Grijalba (teaching assistant) · Alexander Quispe (instructor)

Wednesday, September 23, 2026 · 19:00–22:00

Goal of the session. Compute an information gain by hand and confirm scikit-learn picks the same split; watch a single tree overfit and a bagged ensemble not; check the variance formula $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$ on real trees; reproduce the AdaBoost trace of Lecture 5 to three decimals; and tune the number of boosting rounds the only way that is honest.

Materials

- `lab_03_student.ipynb` / `lab_03_solution.ipynb`.
- Sources: CS 155 Problem Set 3 (decision trees, information gain, `min_samples_leaf`).
- Data: `load_breast_cancer` ($N = 569$, $p = 30$), bundled with scikit-learn.

1 Impurity and Information Gain (35 min)

Task 1 — three impurities, two lecture examples

Write `entropy(p)`, `gini(p)`, `misclass(p)` for a vector of class proportions, and `gain(parent, children, impurity)` where `children` is a list of class-count vectors. Reproduce Lecture 4:

1. Parent (8,4) split into (6,1) and (2,3): gains 0.1685 (entropy), 0.1016 (Gini), 0.0833 (misclassification).
2. Parent (400,400): split A into (300,100), (100,300) vs. split B into (200,400), (200,0). Misclassification ties at 0.25 after either split; Gini gives 0.375 vs. 0.333; entropy 0.811 vs. 0.689.

Task 2 — the root split, by hand

Write `best_split(X, y, impurity)`: for every feature j and every midpoint s between consecutive sorted values of $x_{:,j}$, compute the gain of the split $x_j \leq s$; return the best (j, s, Δ) . Run it on the breast-cancer training set with Gini. Then fit `DecisionTreeClassifier(max_depth=1)` and read `tree_.feature[0]` and `tree_.threshold[0]`. They must agree.

2 One Tree (25 min)

Task 3 — overfitting, in two directions

With an 80/20 split:

1. For `max_depth` $\in \{1, 2, \dots, 12\}$, plot training and test error. Where does the test curve turn?
2. For `min_samples_leaf` $\in \{1, 2, 5, 10, 20, 50\}$ at unlimited depth, same plot. Explain, using Lecture 4's remark that a leaf's prediction has variance at most $\sigma^2/n_{\min}$, why this knob works.
3. Grow the unrestricted tree and report: number of leaves, training error, test error. This is the low-bias, high-variance object that bagging is built for.

3 Bagging and the Variance Formula (45 min)

Task 4 — bagging from scratch

Implement `bagged_trees(X, y, B, seed)`: B bootstrap samples (`rng.integers(0, N, N)`), one unrestricted tree each; return the list of trees and the list of index arrays. Predict by majority vote.

1. Plot test error against $B \in \{1, 2, 5, 10, 25, 50, 100, 200\}$. Overlay the single-tree error from Task 3.
2. Out-of-bag error: for each training point, vote only among trees whose bootstrap sample did not contain it. Compare with the test error and with `BaggingClassifier(oob_score=True)`.
3. Check the e^{-1} claim: the mean fraction of distinct training points per bootstrap sample should be ≈ 0.632 .

Task 5 — measure ρ and test the formula

Treat each tree's prediction on the test set as a ± 1 vector. Compute the average pairwise correlation $\hat{\rho}$ across the $B = 100$ trees and the average per-tree variance $\hat{\sigma}^2$ (across test points). Then compute the variance of the averaged vote directly, and compare with $\hat{\rho}\hat{\sigma}^2 + \frac{1-\hat{\rho}}{B}\hat{\sigma}^2$. Report $\hat{\rho}$: it is the floor bagging cannot go below.

Task 6 — random forests lower ρ

Repeat Task 5 with `RandomForestClassifier(max_features=m)` for $m \in \{1, 3, 5 (\approx \sqrt{30}), 10, 30\}$. Tabulate $\hat{\rho}$, single-tree test error, and forest test error against m . The pattern to see: $\hat{\rho}$ falls as m falls; single trees get worse; the forest gets better — until m is so small that the trees are useless. Finish with permutation importance (`sklearn.inspection.permutation_importance`) on the best forest: which five features matter?

4 Boosting (50 min)

Task 7 — AdaBoost by hand

Implement AdaBoost with decision stumps on a 1-D dataset, base learners $h(x) = \pm \text{sign}(x - t)$ with t at midpoints. Run it on $x = (1, 2, 3, 4, 5)$, $y = (+1, +1, -1, -1, +1)$ for three rounds and print, per round, the weights, the chosen stump, ε_m , α_m , Z_m . Your numbers must match Lecture 5's table: $\varepsilon = (0.2, 0.25, 0.333)$, $\alpha = (0.693, 0.549, 0.347)$, $Z = (0.8, 0.866, 0.943)$, and $\prod Z_m = 0.653$. Then verify the two identities of the proof: $Z_m = 2\sqrt{\varepsilon_m(1 - \varepsilon_m)}$, and $\frac{1}{N} \sum_i e^{-y_i f_3(x_i)} = \prod_m Z_m$.

Task 8 — early stopping on M

On the breast-cancer data, fit `GradientBoostingClassifier(n_estimators=2000, learning_rate= $\nu$ , max_depth=3)` for $\nu \in \{1, 0.1, 0.01\}$. Use `staged_predict_proba` to get the validation log loss after every round on a held-out 20% of the training set. Plot the three curves. For each ν , report the M that minimizes validation loss and the test error of the model truncated there. Which ν wins, and at what cost in M ?

Task 9 — three ensembles, one table

Tune each of bagging, random forest and gradient boosting by 5-fold CV on the training set (a small grid each — B , m , and (ν, M, depth) respectively). Report test accuracy, AUC and wall-clock fit time for the three tuned models and for the single tuned tree. Two sentences on which you would deploy and why.

5 Exercises (to submit before Lab 4)

Exercise 1. Gradient boosting for regression, from scratch: with squared loss and stumps, implement the loop of Lecture 5's algorithm (pseudo-residuals, fit, line search, shrinkage). Check that with $\nu = 1$ it matches `GradientBoostingRegressor(max_depth=1, learning_rate=1)` on the diabetes data after 50 rounds.

Exercise 2. Verify XGBoost's leaf formula in the simplest case: for squared loss, $g_i = f(\mathbf{x}_i) - y_i$ and $k_i = 1$, so with $\lambda = 0$ the optimal leaf value $-G_j/K_j$ is the mean residual in the leaf. Show this numerically on one boosting round, then repeat with $\lambda = 10$ and explain what changed.

Exercise 3. Prove that in AdaBoost the weighted error of h_m under the updated weights $\mathbf{w}^{(m+1)}$ is exactly $\frac{1}{2}$. (Hint: split Z_m into the correct and incorrect parts and use the value of α_m .) Verify it in your Task 7 code. What does this say about why the same stump is never chosen twice in a row?

Submission

One executed notebook, `lab03_lastname_firstname.ipynb`, to the course drive before Wednesday, September 30, 18:00.